Laboratorio 6: LED con PS y PL, simulación de FFT y filtro FIR y consumo de potencia en la Arty Z7-20

Alumno: Ricardo Amiel Acuña Villogas

5 de noviembre de 2026

Resumen

Se implementó en la Arty Z7-20 un sistema que enciende LEDs desde la lógica programable (PL) y desde el procesador ARM (PS), se simularon los núcleos xFFT y FIR Compiler de AMD y se estimó el consumo de potencia de los tres diseños con Vivado 2025.2. El diseño del LED funcionó en la placa: dos LEDs responden al programa en C y dos a la lógica de la FPGA, con un uso menor al 1 % de los recursos. La FFT de 1024 puntos ubicó el tono de 1 MHz en el índice 10 y la transformada inversa reconstruyó la entrada con una similitud de 0.99999. La potencia total estimada fue de 1.401 W para el LED, 1.532 W para la FFT y 1.498 W para el FIR; en los tres casos el PS7 consume cerca de 1.26 W, por lo que el costo propio de la lógica de procesamiento es de solo 7, 127 y 92 mW respectivamente.

Material complementario

- Repositorio con el código, los scripts y este informe: `https://github.com/ricardoamiel/Arty-ZYNQ-7000---Laboratorio-6`
- Video de la placa en funcionamiento: ver video en Drive
- Carpeta con todo el material del laboratorio: carpeta en Drive

1. Introducción

La Arty Z7-20 usa el SoC Zynq-7000 XC7Z020, que integra en un mismo chip un sistema de procesamiento (PS) con dos núcleos ARM Cortex A9 y una lógica programable (PL) equivalente a una FPGA Artix 7. El PS ejecuta software en C y la PL implementa hardware dedicado; ambos se comunican por los buses AXI. Esta división permite repartir una tarea de procesamiento de señales: la PL ejecuta en paralelo las operaciones costosas, como la FFT o un filtro FIR, y el PS controla, configura y almacena los resultados.

El arranque del Zynq sigue dos etapas. Primero la BootROM carga el FSBL, que configura el PS y programa el bitstream en la PL. Luego se ejecuta la aplicación, ya sea un programa standalone en C o un sistema Linux mediante U Boot. En este laboratorio se usa la ruta standalone, programada por JTAG, que es suficiente para controlar periféricos y probar núcleos de procesamiento sin un sistema operativo.

2. Objetivos

1. Diseñar un sistema que encienda LEDs desde la FPGA (PL) y desde el procesador (PS) de la Arty Z7-20.

2. Simular los núcleos de propiedad intelectual xFFT y FIR Compiler de AMD y verificar su respuesta.

3. Estimar y analizar el consumo de potencia de los diseños implementados.

3. Materiales y herramientas

- Placa Digilent Arty Z7-20 (XC7Z020CLG400-1), cable micro USB y jumper JP4 en modo JTAG.
- Vivado ML Standard 2025.2 y Vitis Embedded Development 2025.2 en Ubuntu 24.04, con instalación mínima: solo la familia Zynq-7000 (descarga de 23 GB frente a los 163 GB de una instalación completa).
- Archivos de placa de Digilent (vivado-boards) para la Arty Z7-20.
- Python 3.12 con NumPy y Matplotlib para el posprocesamiento de la simulación.

4. Parte 1: encendido de LEDs con la FPGA y el procesador

4.1. Arquitectura

El diseño de bloques contiene el PS7 configurado con el preajuste de la placa, un AXI GPIO de 2 bits conectado al puerto maestro M_AXI_GP0 y un módulo RTL propio. La Figura 1 resume las conexiones.

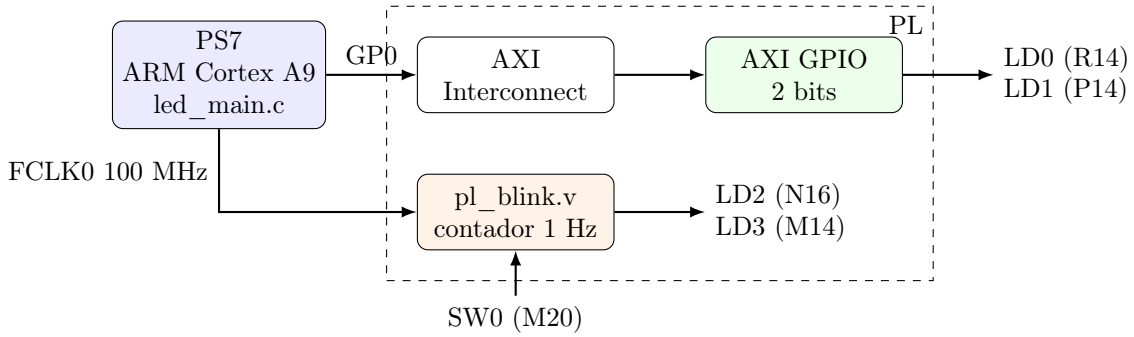


Figura 1: Arquitectura del diseño: LD0 y LD1 los controla el procesador; LD2 y LD3 los controla solo la lógica programable.

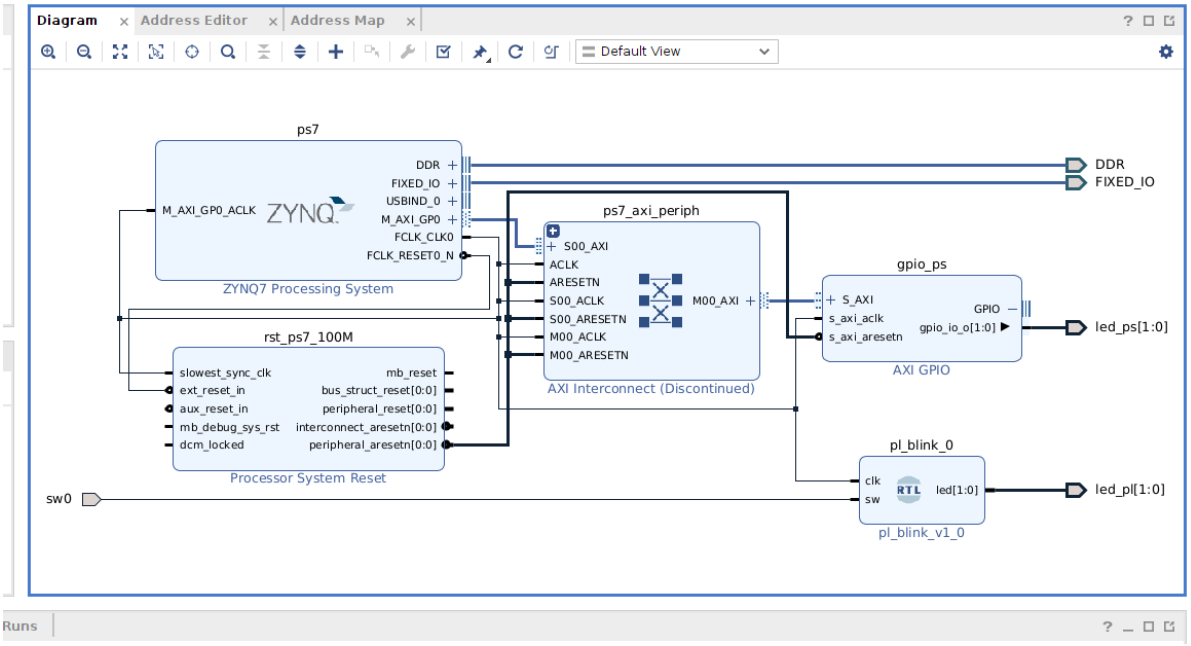


Figura 2: Diseño de bloques generado en Vivado 2025.2: PS7, AXI Interconnect, AXI GPIO (gpio_ps), reinicio del procesador (rst_ps7_100M) y el módulo RTL pl_blink_0.

4.2. Lógica programable (PL)

El módulo pl_blink divide el reloj FCLK0 de 100 MHz con un contador de 32 bits; cada 50 millones de ciclos invierte la salida, de modo que LD2 parpadea a 1 Hz sin intervención del procesador. LD3 copia directamente el estado del switch SW0.

Listing 1: Módulo pl_blink.v

```
// Parte PL: logica pura en la FPGA, sin intervencion del procesador.
// led[0] (LD2) parpadea a 1 Hz con un contador; led[1] (LD3) copia el switch SW0.
module pl_blink #(
    parameter integer CLK_HZ = 100_000_000
)(
    input wire clk,
    input wire sw,
    output wire [1:0] led
);
    localparam integer HALF = CLK_HZ / 2;
    reg [31:0] cnt = 0;
    reg q = 1'b0;
    always @(posedge clk) begin
        if (cnt == HALF - 1) begin
            cnt <= 0;
            q <= ~q;
        end else begin
            cnt <= cnt + 1;
        end
    end
    assign led[0] = q;
    assign led[1] = sw;
endmodule
```

4.3. Sistema de procesamiento (PS)

El programa standalone inicializa el AXI GPIO y escribe cada 500 ms un patrón de dos bits: LD0 cambia en cada iteración y LD1 cada dos, lo que forma un contador binario visible. Cada iteración se reporta por la UART a 115200 baudios.

Listing 2: Programa led_main.c

```
/* Parte PS: el procesador ARM Cortex A9 controla LD0 y LD1 por AXI GPIO. */
#include "xparameters.h"
#include "xgpio.h"
#include "sleep.h"
#include "xil_printf.h"

#ifdef SDT
#define GPIO_ID XPAR_XGPIO_0_BASEADDR
#else
#define GPIO_ID XPAR_GPIO_0_DEVICE_ID
#endif

int main(void)
{
    XGpio leds;
    if (XGpio_Initialize(&leds, GPIO_ID) != XST_SUCCESS) {
        xil_printf("Error al iniciar AXI GPIO\r\n");
        return -1;
    }
    XGpio_SetDataDirection(&leds, 1, 0x0); /* canal 1, todo salida */

    xil_printf("Hola mundo desde el ARM Cortex A9 de la Arty Z7-20\r\n");
    xil_printf("LD0 y LD1 controlados por el procesador\r\n");
    u32 n = 0;
    while (1) {
        /* LD0 se enciende y apaga cada 500 ms; LD1 cambia cada 2 ciclos */
        u32 patron = (n & 1) | (((n >> 1) & 1) << 1);
        XGpio_DiscreteWrite(&leds, 1, patron);
        xil_printf("ciclo %d LD1 LD0 = %d%d\r\n", n, (patron >> 1) & 1, patron & 1);
        n++;
        usleep(500000);
    }
    return 0;
}
```

4.4. Flujo de implementación

1. Vivado crea el diseño de bloques, sintetiza, implementa y genera el bitstream y el archivo de plataforma sys_wrapper.xsa (script led_hw.tcl).

2. Vitis crea la plataforma standalone a partir del .xsa y compila la aplicación (create_vitis_led.py).

3. XSDB programa el bitstream, inicializa el PS con ps7_init y descarga el ejecutable al Cortex A9 (program.tcl).

Los tres pasos se ejecutaron por terminal con run_led.sh (hw, sw y prog). Las Figuras 3, 4 y 5 muestran el resultado en cada herramienta.

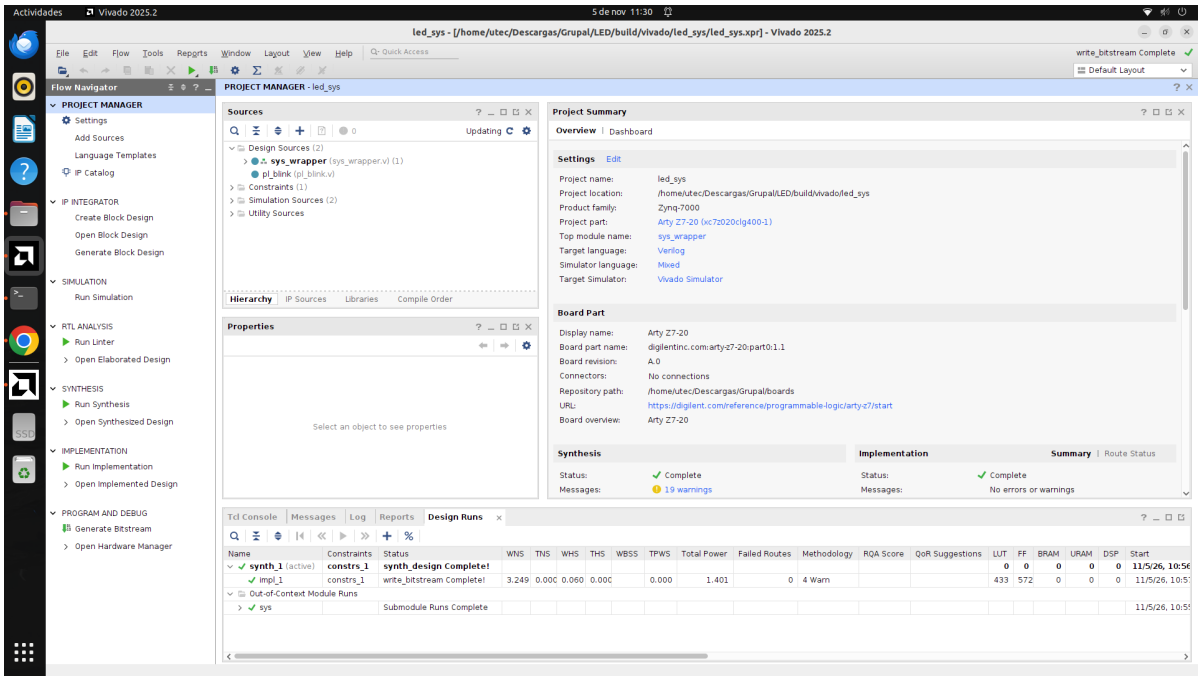


Figura 3: Resumen del proyecto en Vivado: placa Arty Z7-20, síntesis e implementación completas y bitstream generado.

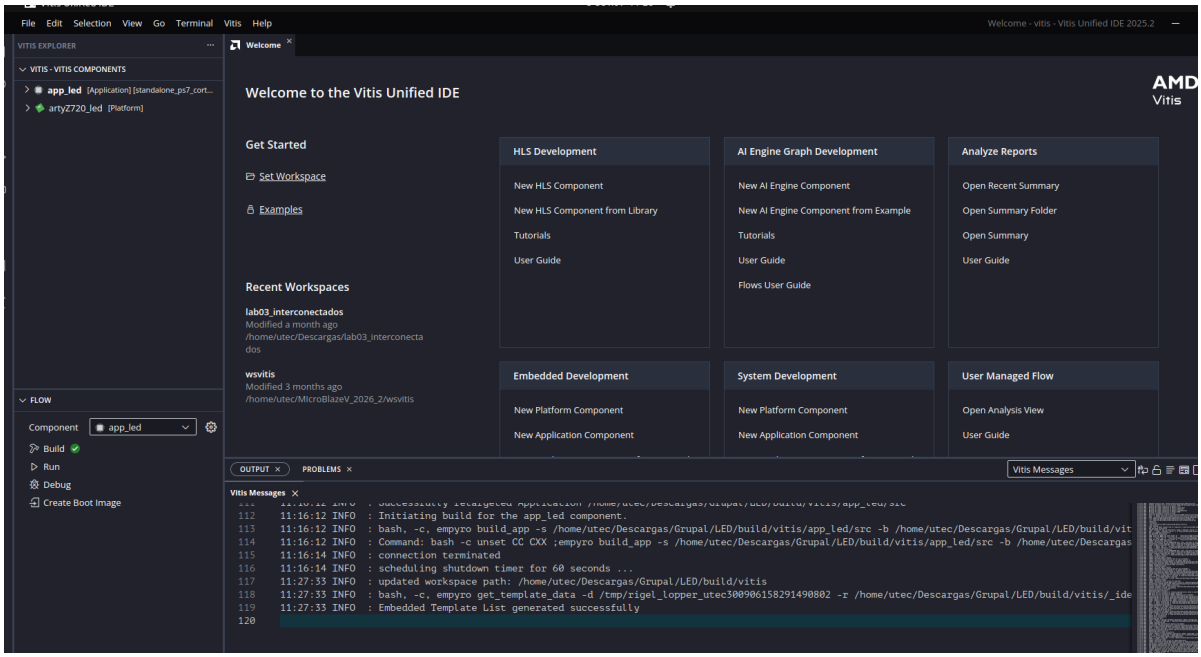


Figura 4: Vitis Unified IDE 2025.2 con la plataforma artyZ720_led y la aplicación app_led compiladas.

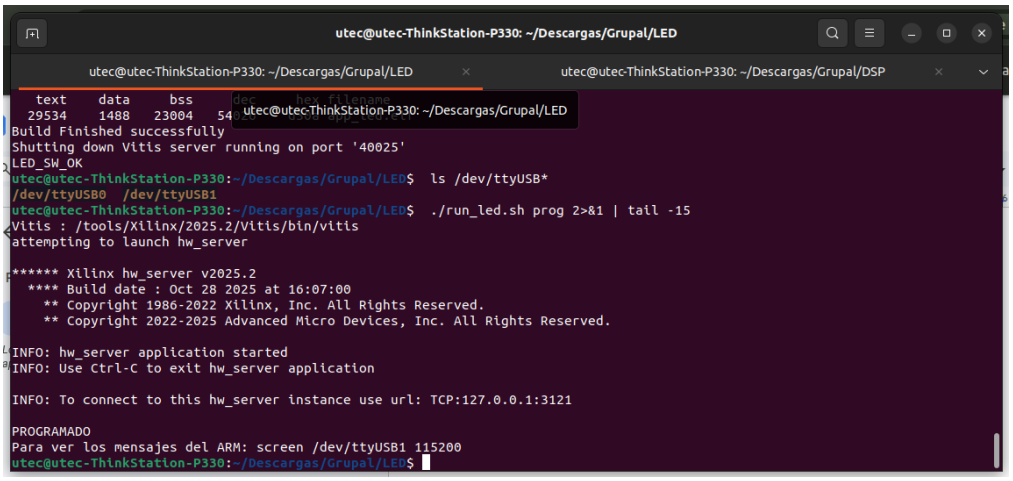


Figura 5: Compilación de la aplicación y programación de la placa por JTAG con xsdb y hw_server 2025.2.

4.5. Resultados

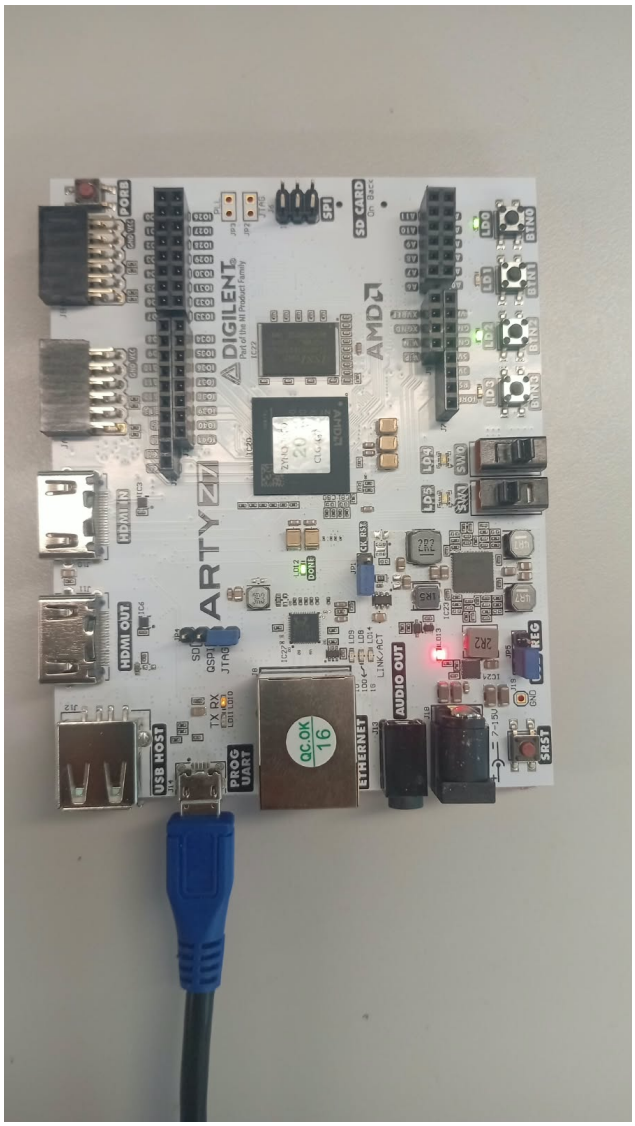


Figura 6: Arty Z7-20 ejecutando el diseño. LD0 encendido por el procesador, LD2 encendido por el contador de la PL, LD3 apagado con SW0 abajo, LD12 (DONE) indica la FPGA configurada y el LED TX muestra la transmisión por la UART.

La consola serial (115200 baudios) muestra la secuencia que escribe el procesador, que coincide con lo observado en LD1 y LD0 (Figura 7).

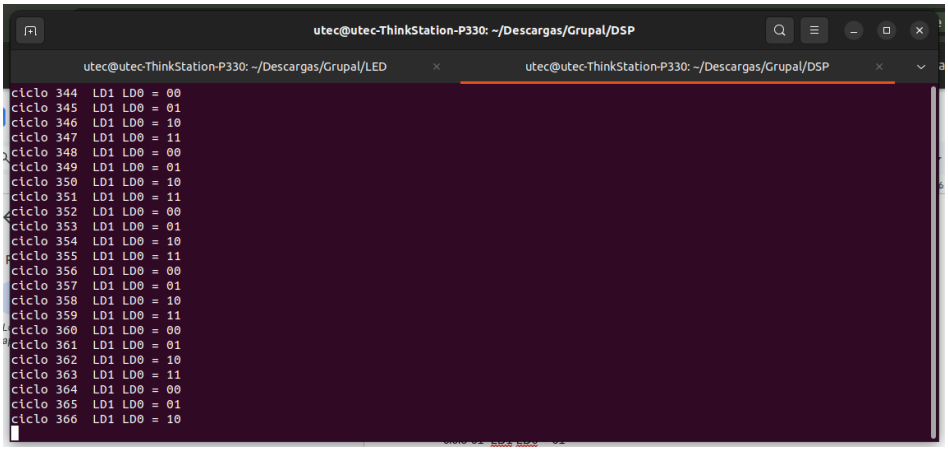


Figura 7: Salida de la UART: el procesador recorre el contador de dos bits 00, 01, 10, 11 cada 500 ms.

El video de la placa en funcionamiento (ver enlaces en Material complementario) muestra el parpadeo de LD2 a 1 Hz independiente del contador del procesador y la respuesta de LD3 al switch SW0.

Cuadro 1: Recursos y tiempos del diseño del LED (Vivado, implementación).

Recurso	Usado	Disponible en XC7Z020
LUT	433	53200
Flip flops	572	106400
BRAM	0	140
DSP	0	220
WNS (holgura de setup)	3.249 ns	
WHS (holgura de hold)	0.060 ns	

El diseño usa menos del 1 % de la lógica disponible y cumple las restricciones de tiempo a 100 MHz, ya que ambas holguras son positivas. La mayor parte de las LUT y flip flops corresponden al AXI Interconnect y al AXI GPIO; el contador de pl_blink solo requiere 33 registros.

5. Parte 2: simulación de la FFT y del filtro FIR

5.1. Configuración de los núcleos

Cuadro 2: Parámetros principales de los núcleos simulados (dsp_sim.tcl).

Núcleo	Parámetro	Valor
xFFT 9.1	Longitud máxima / usada	2048 / 1024 puntos (configurable en ejecución)
	Arquitectura	Pipelined streaming I/O
	Datos de entrada / factores de fase	16 bits / 24 bits
	Orden de salida	Natural
	Reloj	100 MHz
FIR Compiler 7.2	Pasabanda	Coeficientes BPF.coe, 16 bits
	Hilbert	Coeficientes hilbert_0.coe
	Decimador	Factor 4, LPF_dec.coe
	Interpolador	Factor 4, LPF_int.coe
	Arquitectura	Systolic multiply accumulate

5.2. Estímulos

Para la FFT el testbench genera un tono complejo de 1 MHz muestreado a 100 MHz en un bloque de 1024 muestras; el resultado de la transformada directa entra a una segunda instancia configurada como transformada inversa. Para el FIR se inyectan 1024 muestras de ruido aleatorio, cuyo espectro plano permite observar la respuesta en frecuencia de cada filtro.

Con $f_s = 100$ MHz y $N = 1024$, la resolución en frecuencia es

$$\Delta f = \frac{f_s}{N} = \frac{100\ \text{MHz}}{1024} \approx 97.66\ \text{kHz},$$

por lo que el tono de 1 MHz debe aparecer cerca del índice $k = 1\ \text{MHz}/\Delta f \approx 10.24$.

5.3. Resultados de la FFT

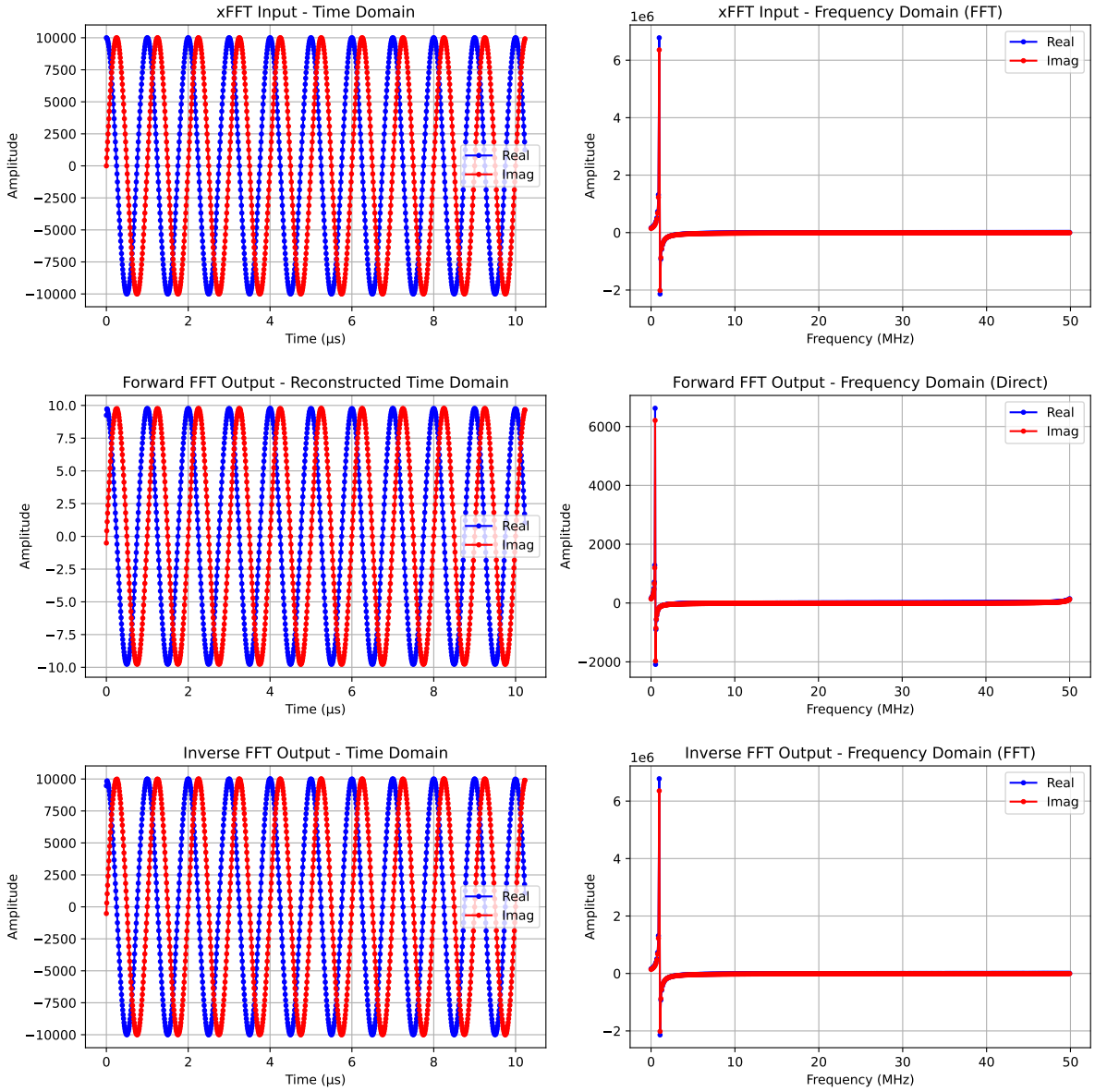


Figura 8: Entrada, salida de la FFT directa y salida de la FFT inversa del núcleo xFFT.

La salida directa concentra la energía en el índice $k = 10$, que corresponde a $10 \cdot 97.66 \approx 976.6$ kHz, el bin más cercano a 1 MHz. Como el tono cae en $k = 10.24$ y no en un entero, parte de la energía se reparte en los bins vecinos $k = 11$ y $k = 9$ (magnitudes de 2869 y 1755 frente a 9076 en el pico); esta es la fuga espectral propia de una ventana rectangular. El núcleo escala la salida por $1/1024$ para evitar desbordes en sus 16 bits: la relación entre el pico del hardware y el de numpy.fft es 0.00098, es decir, exactamente $1/N$.

La transformada inversa recupera la señal original: su amplitud máxima es 10054 frente a 10000 en la entrada y la similitud normalizada entre ambas señales es 0.99999. La pequeña diferencia proviene del redondeo en aritmética de punto fijo dentro de las dos transformadas.

Cuadro 3: Verificación numérica de la FFT (datos de la simulación frente a numpy).

Magnitud	Valor
Índice del pico (hardware y numpy)	10
Bins vecinos con fuga	11 y 9
Escala del hardware respecto a numpy	$0.00098 \approx 1/1024$
Similitud entrada frente a IFFT	0.99999

5.4. Resultados del filtro FIR

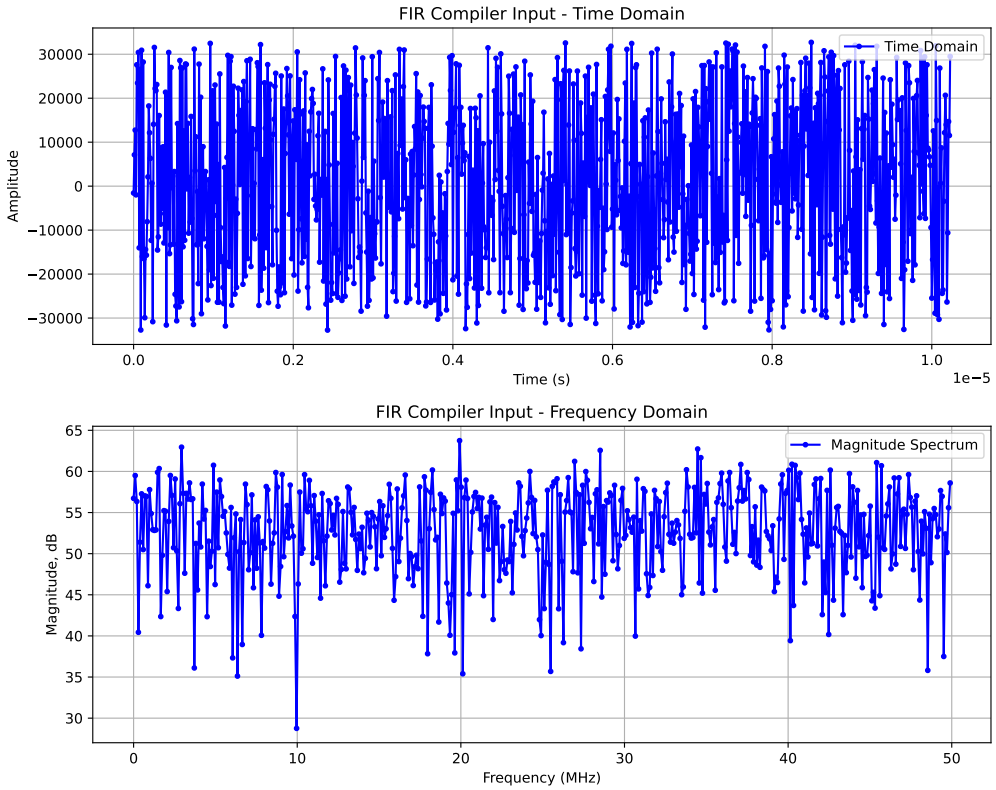


Figura 9: Señal de entrada (ruido) al FIR Compiler.

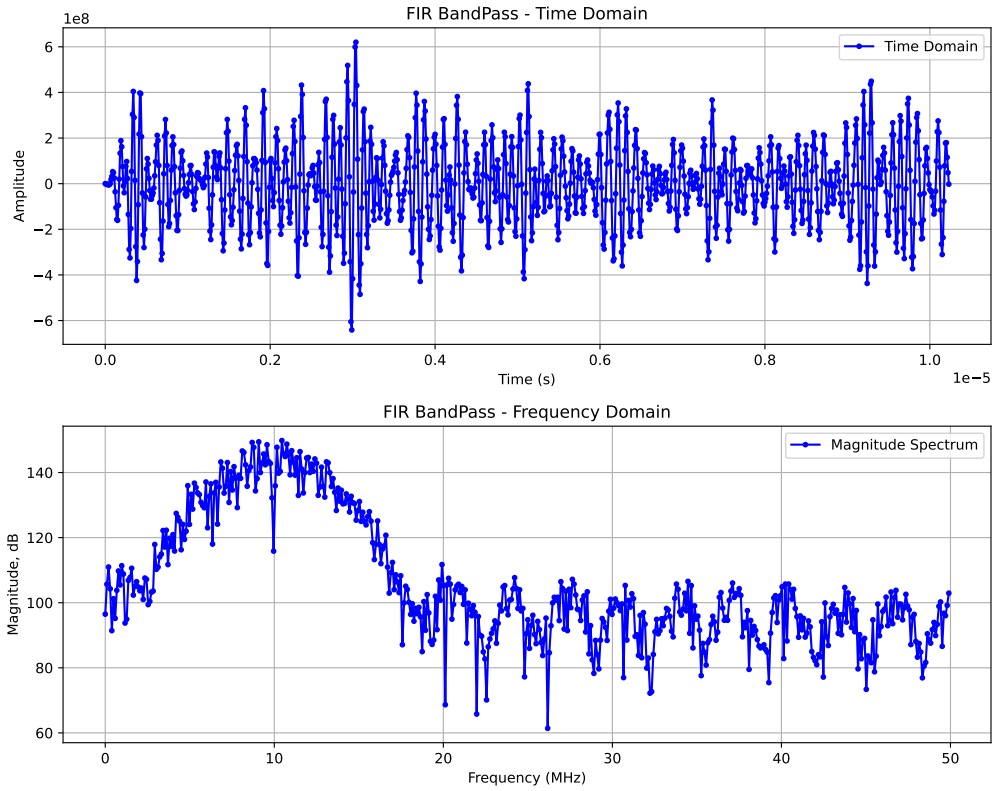


Figura 10: Salida del filtro pasabanda.

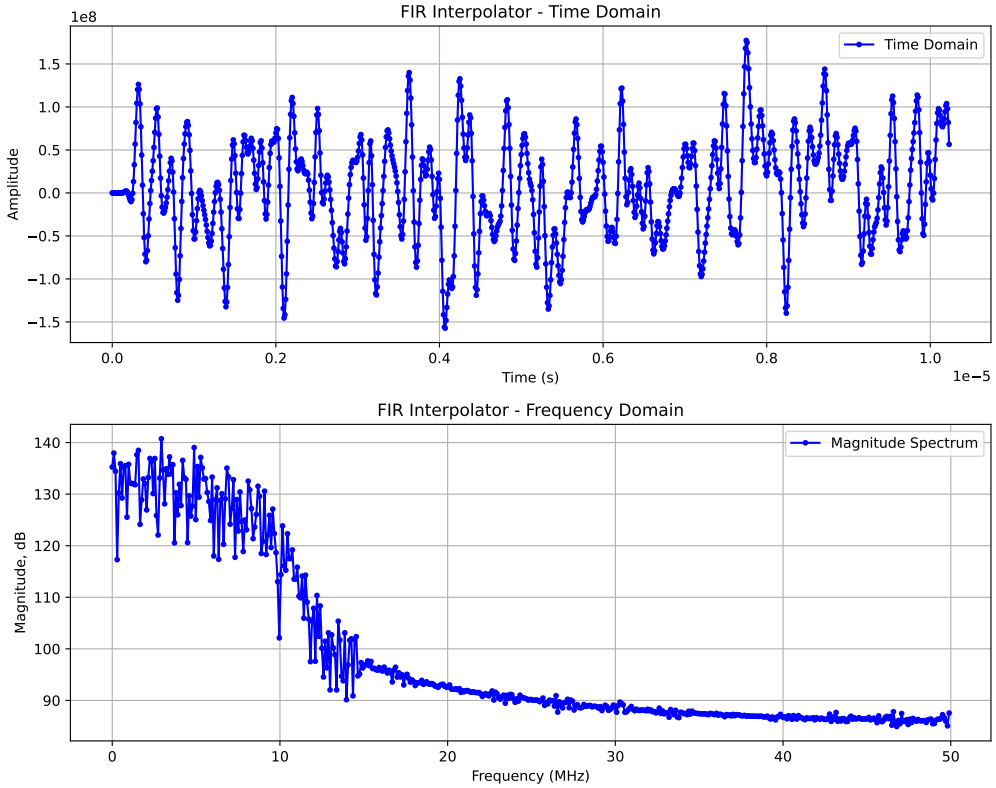


Figura 11: Salida del interpolador.

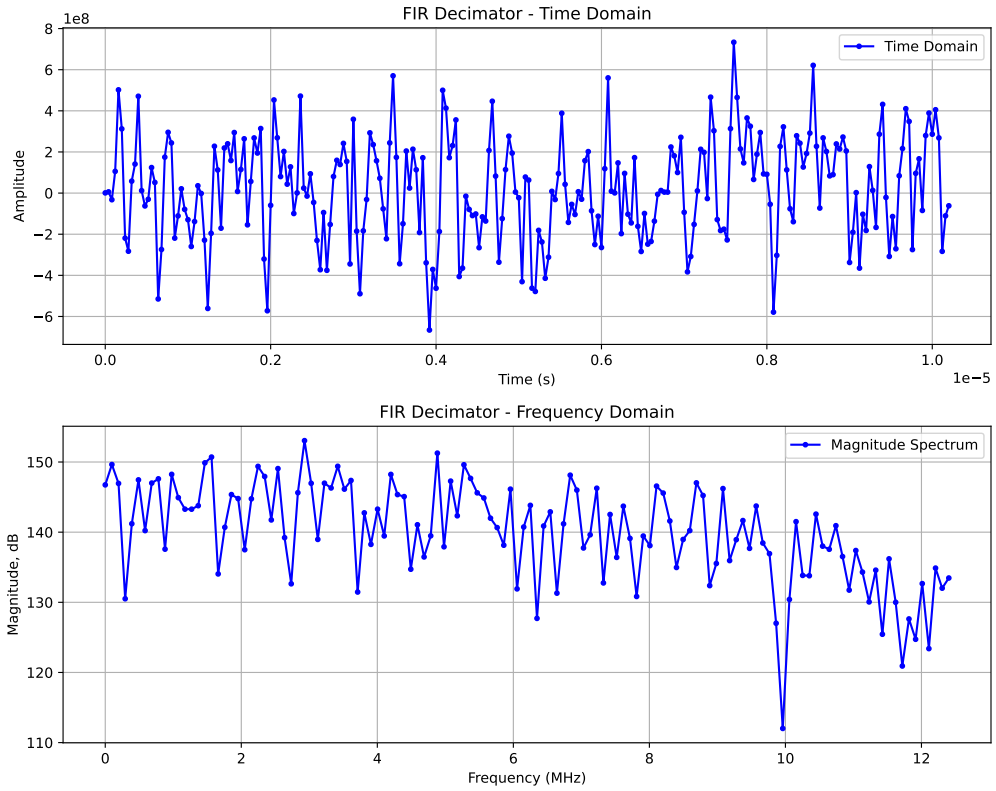


Figura 12: Salida del decimador.

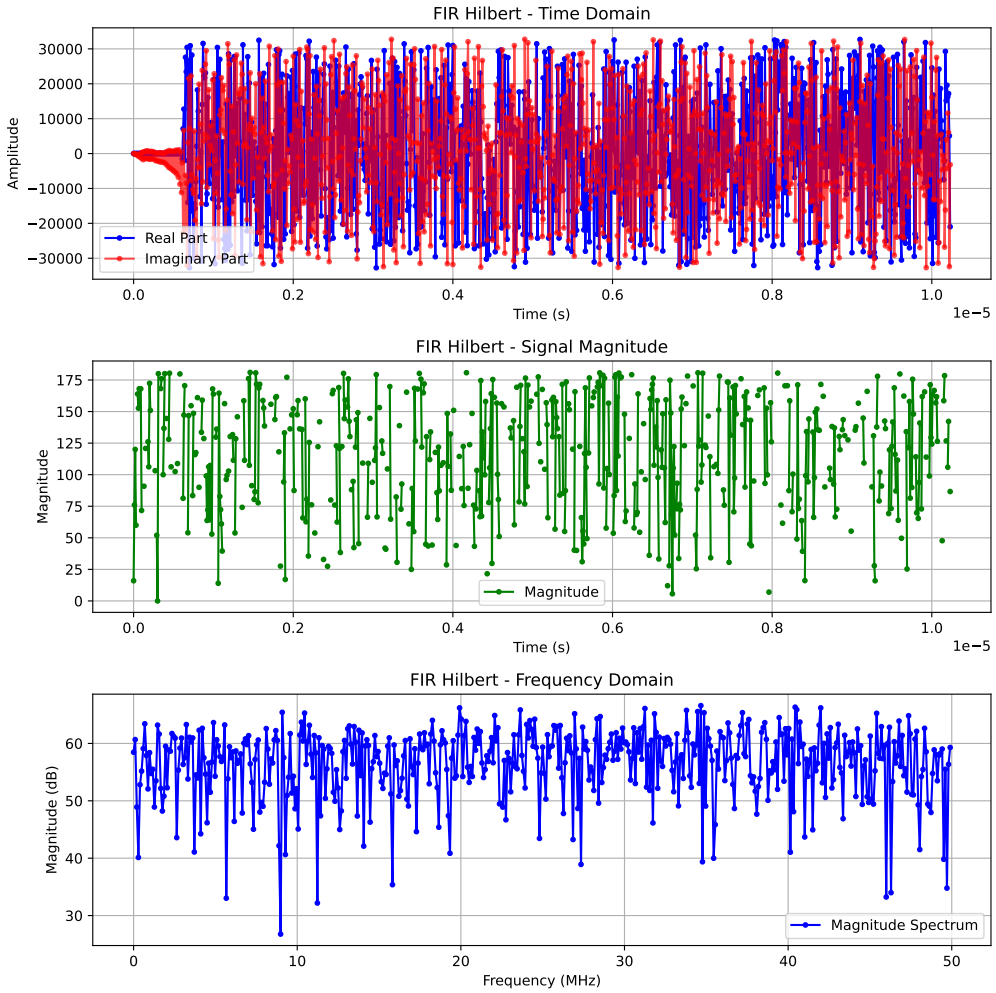


Figura 13: Salida del transformador de Hilbert (parte real e imaginaria).

- Entrada: el ruido aleatorio tiene un espectro aproximadamente plano entre 0 y 50 MHz, por lo que sirve para observar la respuesta de cada filtro.
- Pasabanda: conserva la banda centrada en 10 MHz; el 90 % de la energía de salida queda entre 6.9 y 13.1 MHz, y fuera de ella la atenuación es de unos 40 a 50 dB.
- Interpolador por 4: el filtro paso bajo elimina las imágenes espectrales que aparecen al insertar ceros; la salida conserva la banda baja hasta unos 10 MHz y atenúa el resto en más de 40 dB.
- Decimador por 4: filtra antes de submuestrear y la salida queda a 25 MHz con 256 muestras (1024 entre 4); su espectro ocupa de 0 a 12.5 MHz sin componentes plegadas visibles.
- Hilbert: entrega una parte real y una imaginaria desfasadas 90 grados para formar la señal analítica. Con ruido real a la entrada la supresión de frecuencias negativas medida es de solo 4.9 dB, porque el filtro solo actúa dentro de su banda de paso; con una señal de banda angosta el efecto sería más marcado.

6. Parte 3: consumo de potencia

6.1. Metodología

La potencia se estimó con report_power de Vivado sobre cada diseño implementado, con las condiciones por defecto (temperatura ambiente de 25 °C y actividad de conmutación vectorless de

12.5 %). La potencia total se divide en estática, debida a las corrientes de fuga del chip, y dinámica, proporcional a la frecuencia y a la actividad:

$$P_{din} = \alpha\, C\, V^2 f,$$

donde α es el factor de actividad, C la capacitancia conmutada, V la tensión de alimentación y f la frecuencia de reloj.

6.2. Resultados

Cuadro 4: Recursos usados por cada diseño implementado (XC7Z020).

Recurso	LED	FFT	FIR
LUT (de 53200)	433 (0.8 %)	7660 (14.4 %)	6265 (11.8 %)
Flip flops (de 106400)	572 (0.5 %)	11736 (11.0 %)	11474 (10.8 %)
BRAM (de 140)	0	17 (12.1 %)	8.5 (6.1 %)
DSP (de 220)	0	20 (9.1 %)	57 (25.9 %)

Cuadro 5: Resumen de potencia estimada por Vivado (report_power).

Parámetro	LED (PS + PL)	FFT (xFFT + DMA)	FIR (4 filtros + DMA)
Potencia total en chip (W)	1.401	1.532	1.498
Potencia dinámica (W)	1.263	1.393	1.358
Potencia estática (W)	0.137	0.139	0.141
Temperatura de unión (°C)	41.2	42.7	42.3

Cuadro 6: Potencia dinámica por componente (W).

Componente	LED (PS + PL)	FFT (xFFT + DMA)	FIR (4 filtros + DMA)
Clocks	0.003	0.042	0.038
Slice Logic	<0.001	0.021	0.006
Signals	0.001	0.026	0.010
Block RAM	0	0.021	0.003
DSPs	0	0.018	0.034
I/O	0.002	0	<0.001
PS7 (procesador)	1.256	1.266	1.266

Cuadro 7: Potencia dinámica atribuible solo a la lógica programable (dinámica total menos PS7).

	LED (PS + PL)	FFT (xFFT + DMA)	FIR (4 filtros + DMA)
PL dinámica (W)	0.007	0.127	0.092

6.3. Análisis

1. El PS7 domina el consumo. En los tres diseños el procesador aporta entre 1.256 y 1.266 W, más del 90 % de la potencia total. Esa cifra incluye los dos núcleos Cortex A9, la memoria DDR3 y los periféricos fijos, y se mantiene casi igual porque la configuración del PS es la misma en todos los casos.

2. La lógica del LED es casi gratuita. El contador y el AXI GPIO agregan solo 7 mW de potencia dinámica; la potencia estática de 0.137 W existe aunque la lógica no conmute.

3. La FFT es el diseño de PL más costoso (127 mW). Necesita 17 bloques BRAM para guardar los datos y los factores de fase de 2048 puntos, y su reloj llega a muchos más registros; por eso Clocks, Signals y Block RAM superan a los del FIR.

4. El FIR usa más DSP pero consume menos (92 mW). Sus cuatro filtros ocupan 57 DSP (26 % del chip) en estructura sistólica, y los DSP son bloques de multiplicación dedicados más eficientes que la lógica general; además necesita la mitad de BRAM que la FFT.

5. La temperatura de unión se mantiene entre 41 y 43 °C, muy por debajo del límite, con un margen de temperatura ambiente cercano a 68 °C.

6. Los diseños de la FFT y del FIR incluyen el DMA y un System ILA para depuración por JTAG; el ILA usa BRAM y lógica que no serían necesarias en un diseño final, así que su consumo real sería algo menor.

Para reducir el consumo se puede: bajar la frecuencia de FCLK0 o de los procesadores cuando la tarea lo permita, ya que la potencia dinámica es proporcional a f; deshabilitar los periféricos del PS que no se usan (USB, Ethernet, SD); retirar el ILA de la versión final; y reducir el ancho de los datos o la longitud máxima de la FFT a lo estrictamente necesario, lo que disminuye BRAM, DSP y capacitancia conmutada. Las cifras son estimaciones vectorless con actividad por defecto, con nivel de confianza alto en la implementación pero no medidas en la placa.

7. Conclusiones

1. El Zynq permite controlar el mismo tipo de periférico desde dos dominios: los LEDs de la PL funcionan sin software y con tiempo exacto, mientras que los del PS se controlan con un programa en C a través del bus AXI; ambos convivieron en la placa sin interferencia.

2. Los núcleos xFFT y FIR Compiler responden según lo esperado: la FFT ubica el tono en el bin correcto, muestra la fuga espectral prevista para un tono fuera de bin y la IFFT reconstruye la señal; cada filtro FIR moldea el espectro del ruido según su tipo.

3. El consumo está dominado por el procesador (cerca de 1.26 W); la lógica de procesamiento añade entre 7 y 127 mW según el diseño, por lo que mover cálculos intensivos a la PL tiene un costo energético bajo frente al del PS.

4. La instalación mínima de Vivado y Vitis 2025.2, limitada a la familia Zynq 7000, fue suficiente para todo el laboratorio y redujo la descarga de 163 GB a 23 GB.

Referencias

[1] AMD, Zynq 7000 SoC Technical Reference Manual (UG585).

[2] AMD, Fast Fourier Transform LogiCORE IP Product Guide (PG109).

[3] AMD, FIR Compiler LogiCORE IP Product Guide (PG149).

[4] AMD, Vivado Design Suite User Guide: Power Analysis and Optimization (UG907).

[5] Digilent, Arty Z7 Reference Manual.